

请参阅此出版物的讨论、统计数据和作者简介：<https://www.researchgate.net/publication/232621717>

Git 挖矿的利弊

文章 · 2009年5月

DOI: 10.1109/MSR.2009.5069475 · 来源: DBLP

引用

282

阅读

1,555

6位作者，包括：



克里斯蒂安·伯德

微软

93出版物

8,322引用

[查看简介](#)



厄尔·T·巴尔

伦敦大学学院

108出版物7,778引用

[查看简介](#)



戴维·J·汉密尔顿

加州大学戴维斯分校

5出版物463引用

[查看简介](#)



普雷姆库马尔·德万布

加州大学戴维斯分校

232出版物15,999引用

[查看简介](#)

Git 挖矿的利弊

克里斯蒂安·伯德*, 彼得·C·里格比†, 厄尔·T·巴尔*, 大卫·J·汉密尔顿*, 丹尼尔·M·赫尔曼†, 普雷姆·德万布*

*美国加州大学戴维斯分校
†加拿大维多利亚大学

{鸟, 巴尔, hamiltod, devanbu}@cs.ucdavis.edu {pcr, dmg}@cs.uvic.ca

抽象的

我们现在正见证着去中心化源代码管理 (DSCM) 系统的快速发展, 每个开发者都拥有自己的代码库。DSCM 促进了一种协作模式, 工作成果可以在协作者之间横向 (且私密地) 流动, 而不是总是通过中央代码库上下 (且公开地) 传递。去中心化带来了以下两个方面: 承诺新数据和危险避免对其的误解。我们专注于 Git, 这是一个在备受瞩目的项目中使用非常流行的数据仓库管理 (DSCM)。Git 的去中心化以及其其他功能 (例如自动记录贡献者归属) 带来了更丰富的内容历史记录, 也引发了一些新的问题, 例如“贡献如何在开发者之间流向官方项目存储库?” 然而, 这也存在一些陷阱。提交可能会被重新排序、删除,

或在存储库之间移动时进行编辑。语义

SCM 和 DSCM 中一些常用术语有时会有显著差异, 这可能会导致混淆。例如, 在集中式 CM 中, 所有开发人员都可以立即看到省略, 但在 DSCM 中则无法看到。我们的目标是帮助对 DSCM 感兴趣的研究人员在研究和分析 Git 数据时避免这些及其他风险。

我在疲惫的土地上, 从一根划伤手的
茎上拧出它。

AE Housman, 什罗普郡小伙子

1. 介绍

自本世纪初以来, 研究人员充分利用了 SCM 存储库中已免费提供给开源软件 (OSS) 项目的数据。这些数据已被用来重建软件的创建过程 [1], [2]。研究人员还利用这些数据预测了推荐系统 [3], [4]。

一个 [5], 研究进化模式 [6], [7], [8], 预测错误 [9], [0], [1][11], 并检查协作 [12], [13], [14]。

使用 DSCM 的软件项目数量已经增加, 并且看起来还将继续增长。图1显示

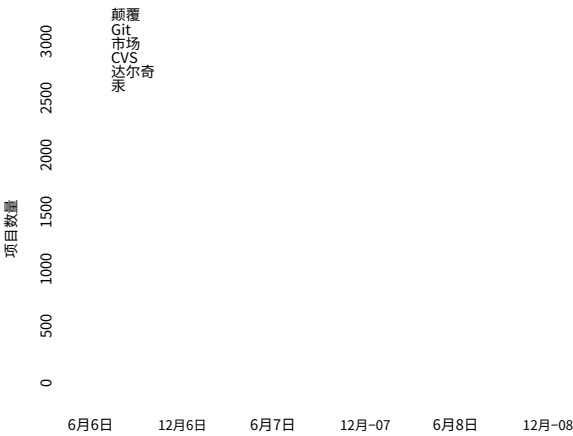


图 1. Debian 项目对 SCM 的使用。

一段时间内使用给定 SCM 的 Debian pack 报告的年龄项目数量¹。由于 36% 的软件包包含 SCM 2009年2月, 信息 (尽管不完整), 该数据有力地表明信息。虽然 git 的使用率仅次于 SVN, 事实上, git 也 git 的使用正在被许多 OSS 项目采用, 例如 X.org、Ruby 增长。on Rail Perl 和 Glasgow Haskell 高调的, 葡萄酒, 桑巴舞, Compiler。

这些和其他项目的存储库对研究人员来 cts 是有趣的重说, 但它们的数据在重要性上与集中式 要方式副手。

Massey 和 Packard [15] 提出了将 CSCM 编辑方法转换为 git 进行数据挖掘的方法。目前, 只然而, 对于我们来说有一篇论文 [16] 引用了基于 git 的项目的数据挖掘测试。这篇论文对从 Linux gi 中提取的数的结果据进行了分析, 并在 Linux 使用从 Git 中挖 t 存储库。我们掘的数据来追踪从仓库到稳定主线 Linux 每周新闻那树的路径 [17]。无论是论文还是官方文 找到他们的档, 都没有明确 Git 和 CE 之间的核心区 m 子系统 git 文章地址

1. 根据使用 Debian 软件包描述中引入的标头的项 这 VC- (SCM-) 目提供的数据库 2006。

这些差异导致了供应链管理 (SCM) 和数据驱动供应链管理 (DSCM) 实践的差异。DSCM 数据中可以观察到全新的现象，并引发了许多新的问题。这些数据为研究人员提供了许多新机遇，但也充满了概念和实践风险。在数据挖掘和分析过程中，我们遭遇了许多挫折，并得出了错误的初步结论，原因是我们对数据以及导致我们观察到的数据的过程理解不完整。

我们比较了 SVN 和 git，它们是集中式和分散式两种流行且具有代表性的 SCM。两者之间最根本的区别在于，在 git 下，开发人员可以以可管理的方式共享代码，而无需使用主存储库，因为主存储库的提交会影响所有其他开发人员。这是因为开发人员拥有自己的存储库，每个存储库都可以讲述软件项目的不同部分。

承诺1：由于 git 项目的任何开发人员都可以公开访问自己的存储库，因此可以恢复更多历史记录，包括正在进行的工作和从未进入稳定代码库的工作。

任何人都可以将自己的 Git 仓库公开，并且目前已经有免费的 Git 托管服务，例如 GitHub、Gitorious 和 repo.or.cz。例如，截至本文撰写时，已有 481 个从 Ruby on Rails 克隆而来的可公开访问的开发者 Git 仓库。²

不同的开发者在其仓库中可以拥有不同的内容。例如，Ubuntu 维护着自己的 Linux 内核仓库，其中包含他们自己的更改，其中一些更改从未被纳入官方内核树。Linux 开发者 David Miller 维护着一个名为 net-next-2.6 其中包含新的或实验性的代码，并且是内核中网络代码的暂存区。

尽管采用去中心化模式，许多开源软件项目都有一个“官方”仓库，用于发布新版本，并供开发者使用。这些仓库，以及更活跃开发者的仓库，很可能是研究人员最感兴趣的。

在使用 SVN 的项目中，提交通常只有在经过审核后才会被录入仓库；其余活动（例如失败的尝试、实验）基本上不可见。使用 Git，通过挖掘开发人员的仓库，可以更全面地了解开发过程，包括未经打磨的实验性工作。不是使其成为稳定的代码库。

我们开始讨论挖掘 git 的前景和风险，并澄清集中式和分散式 SCM 世界之间的概念差异。

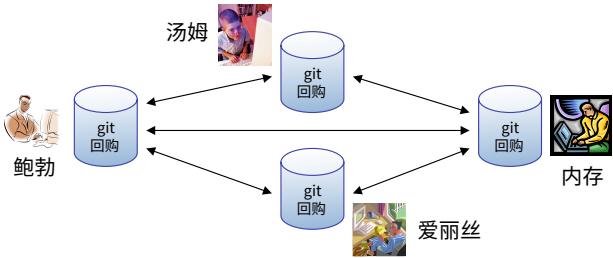


图 2. git 的去中心化使用示例

2.概念差异

危险1：Git 命名法与集中式 SCM (CSCM) 不同：a) 类似的操作有不同的命令；b) 共享术语可能具有不同的含义。

图 2 展示了一个由 Tom、Bob、Alice 和 Ram 组成的 4 人团队的 DSCM 世界，他们合作完成一个大型项目。

无中心 已实现的存储库：每个开发人员有自己的，他们每次使用时进行提交，检查不同的版本等等。s，公司 重新版本差异，

这与集中式 SCM 世界截然不同并产生了命名上的差异，即使是对 git 有所了解的人也会感到困惑；尤其是那些语义上略有不同的术语。为了引导我们的分析中，我们在此大致描述这些差异，并根据需要在具体前景和风险的背景下深入探讨更细微的差异。

在 SVN 和 git 中，*工作副本*是存储库的当前签出状态。开发人员 Alice 执行svn 检出或者git 克隆创建工作副本。Alice 现在可以工作了，当她准备好时，*犯罪*她的贡献，利用犯罪在 SVN 和 git 中都可以使用这个命令。根本区别在于，SVN 中的提交会被发送到中央仓库，而 git 中的提交则保留在本地。因此，SVN 的提交对所有开发人员可见；而 git 的提交则可能不可见。在 SVN 中，Ram 必须发出svn更新查看 Alice 刚刚提交的更改。在 Git 下，只有当以下情况发生时，Alice 的提交才会移动到另一个仓库：a) 另一个开发人员；例如Ram 通过以下方式将更改拉入其存储库的当前分支git pull或者 b) Alice 使用以下方式将她的更改推送到远程存储库git推送。

git 拉取操作与 SVN 更新不同，开发者可以从任意数量的远程仓库拉取到自己的仓库。对于 git 开发者来说，拉取操作比推送操作更常见，因为推送需要写入权限，而且每个开发者都“拥有”自己的仓库，并决定将哪些内容放入其中。不过，git 支持通过裸存储库。裸存储库不是

2. 据 Github.com 2009 年 2 月报道。

由单个人独家拥有，并且没有推送会破坏的工作副本。

在 SVN 中，提交由序列号标识，确定其顺序很简单。在 git 中，每个提交由其内容的 SHA-1 哈希值标识，如果只有两个 git 提交标识符，则无法确定哪一个提交更早执行。

CSCM 中的分支通常用于发布版本和重大的实验性工作。我们观察到，使用 CSCM 的开发人员很少会仅为自己的工作创建分支，并定期将其合并到主线中。相比之下，git 中的分支非常轻量。使用 git 的开发人员可以自由地为新功能或错误修复创建分支，进行测试，然后在相当有信心后将其合并到发布/稳定分支中。git 的一个重要特性是它能够跟踪分支的合并，这是 SVN 所缺乏的。Git 还会创建许多分支，这是去中心化的一个意外副作用（参见“风险 2”，其中讨论了隐式分支）。这种分支与 CSCM 中常见的分支用法有着根本的不同。头任何分支的提交，无论是在 SVN 还是 git 中，都是对该分支的最后一次提交。出于挖矿目的，SVN 和 git 标签是相同的。所有 git 仓库都以一个显式的分支开始，其名称为掌握按照惯例。Git 的掌握类似但不等同于树干在 svn 中，参见危险 3。

由于 git 同时记录了分支和合并的信息，因此与 SVN 中树状的提交结构相比，仓库的历史记录以提交图的形式呈现。在 git 中，一个提交可能有多个父提交（由于合并），也可能是多个提交的父提交（由于分支）。由于任何提交都不可能拥有自己的祖先，因此由提交及其有向父子关系表示的图是有向无环图 (DAG)。DAG 一词贯穿本文，在 git 术语中很常见。不同仓库中的两个 git 提交具有相同的 SHA-1 值，当且仅当它们在 DAG 中具有相同的祖先和相同的内容，这意味着它们的工作副本在创建时是相同的。SHA-1 是一个高度可靠的内容等值指标，可以跟踪从共同来源提取的内容，这使得两个分别从第三个仓库提取相同内容的仓库可以相互合并，而无需使用第三个仓库。

git 中的合并会创建一个新的提交（DAG 中合并的标记），该提交至少有两个父提交。git 和 SVN 中的合并有两个主要区别：1) n 双向合并发生在 git 中，并创建一个合并提交 n 父母；2) SVN 不会明确跟踪合并信息，例如冲突及其解决方案。当发生合并冲突时，git 会将解决方案写入其提交。此外，SVN 开发人员更新其工作副本时发生的合并永远不会被记录，而

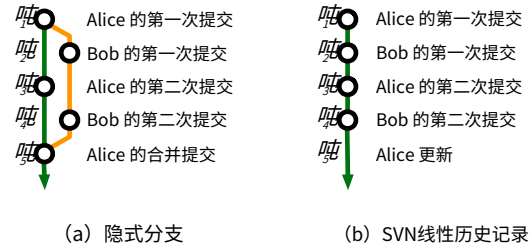


图 3. 相同提交序列的结果。

git 中的类似操作也一样。我们注意到，Perforce、ClearCase、Accurev 和 SVN 1.5 版等工具都包含合并跟踪功能。然而，这些工具创建的 DAG 在语义上有所不同，而且这些工具在开源软件 (OSS) 项目中的流行度和使用历史都很短（SVN 1.5 仅跟踪代码库切换到 1.5 版后发生的合并，无法恢复历史合并）。

危险2：这里有“隐式分支！”

(向龙们致歉)

开发人员可以明确创建明确的 Git 和 SVN 中的分支。SVN 中的所有分支都是显式创建的，但 Git 并非如此。当两个协作的开发人员提交到他们本地的 Git 仓库时，他们的仓库就会出现分歧——~~每个~~每个仓库都包含另一个仓库中不存在的新提交。当一个开发人员从另一个仓库拉取代码时，git 会将远程提交序列合并到拉取代码的开发人员的仓库中。这些更改被记录为两个分支，它们始于两个仓库开始出现分歧的提交（最后一个共同的提交），结束于 git 创建的合并提交。一个分支包含本地开发人员的提交；另一个分支包含来自远程开发人员的提交。两个开发人员都没有明确创建分支。因此，我们称拉取代码可以创建分支~~隐式~~分支对于那些只熟悉 SVN 等集中式 SCM 的人来说，这些分支可能会造成混淆，因为在 SVN 中，分支始终是明确的，并且指的是替代的开发路线。

图 3(a) 展示了隐式分支。在时间之前~~的~~ Alice 和 Bob 的存储库完全相同。之后~~的~~，他们的代码库已经分叉：各自包含对方未知的提交。Alice 和 Bob 继续在各自的代码线上独立工作，如图所示。~~的~~ Alice 从 Bob 的仓库中提取更改并解决所有冲突，合并两个仓库，并自动为合并创建提交。之前~~的~~，Bob 的更改对 Alice 不可见；之后~~的~~ Bob 的代码线是 Alice 历史记录中的一个隐式分支。然而，Bob 的仓库并不包含 Alice 的提交。相比之下，图 3(b) 显示的是在 Alice 和 Bob 使用 CSCM 进行相同更改时形成的历史记录。由于 Bob 的提交更改了共享的中央仓库，因此 Alice 必须更新她的工作副本~~的~~并在她做出决定之前解决任何出现的冲突

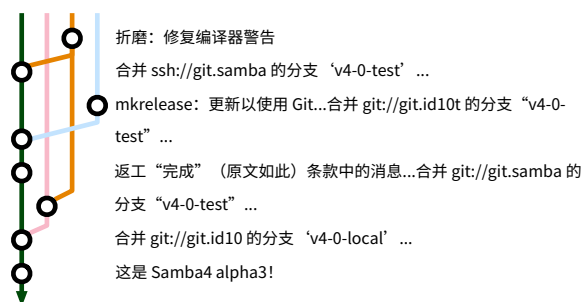


图 4. Samba 4.0 alpha 3 发布前的提交 git DAG 子图。

提交；在图 3(a) 中，Alice 自由地将代码提交到她的本地存储库。同样，Bob 必须在第二次提交之前进行更新并解决潜在的冲突。在 SVN 中，Alice 和 Bob 分别在不同的开发线上工作的事实被忽略了。

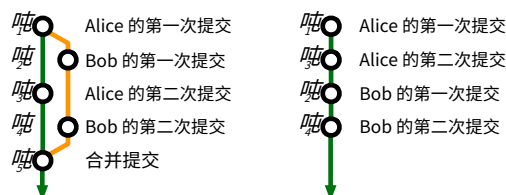
3. Git 数据矿石

承诺2：Git 通过多个存储库的 DAG 中的提交、分支和合并信息，方便恢复更丰富的项目历史记录。

由于 git 同时跟踪仓库内部和仓库之间的隐式和显式分支及合并，因此它能够跟踪 SVN 无法跟踪的数据（SVN 跟踪中央仓库内的分支，但不跟踪合并）。这包括：

- 1) 隐式分支，显示开发人员从其他存储库提取和推送更改的频率；
- 2) 功能（显式）分支，显示协作活动：开发人员之间直接拉取的变更，*对比*通过中间“官方”存储库。
- 3) 合并点，包括冲突文件集，以及谁/何时执行合并和解决。
- 4) 从远程存储库拉取，以及“拉取网络”的整体拓扑。
- 5) DAG，以及同一项目不同仓库中的提交集，可以确定彼此之间的差异和“距离”。

这些数据有很多可能的用途。例如，自发形成的“拉式网络”代码库是分层的、集中式的还是分散式的？在各自分支上开发功能的开发人员之间是否存在可辨别的协作模式？项目内部状态与开发人员的更改集成到官方代码库的频率或速度之间是否存在关联？



(a) Rebase 之前

(b) Rebase 之后

图 5. 变基示例

危险3：Git 没有主线，因此必须适当修改分析方法以将 DAG 考虑在内。

在 SVN 中，一次提交的祖先可以以线性提交序列的形式捕获，主干或“主线”通常也以这种方式建模。Git 项目开发并非遵循单一的“主线”，而是通过 DAG 中的一系列路径，从初始提交到头部。因此，分析方法和存储技术必须处理非线性的提交祖先，包括远程祖先。图 4 描绘了 Samba 存储库在发布之前发生的分支，展现了这种现象。

在 Git 和 SVN 中，开发人员可以并行工作。一系列提交中的两个功能可以同时创建。然而，由于 SVN 要求开发人员更新在提交之前，这两个功能的开发历史将会交错。实际上，在“主线”上并行开发会导致所有工作被投影到一条按日期排序的线上。虽然矿工可以根据需要查看此投影，但这并非强制要求，必须在使用现有分析方法之前创建投影，否则必须使用其他方法。

危险4：Git 历史是修正主义的：存储库所有者可以重写它。

虽然其他 SCM（例如 SVN）也允许重写历史记录，但这很困难，而且很少这样做。Git 允许用户通过名为 *重新定基* 用户选择要变基的提交序列；然后，他可以更改提交的顺序、移除提交、将多个提交中的编辑压缩为一个提交，或将多个分支上的一系列提交展平到单个分支。当提交被重新排序或展平时，所有关于提交内容的信息（日期、作者、文件更改）都会保留，但 DAG 和提交哈希会被修改。最常见的变基用例是展平一系列隐式分支。许多项目都有限制使用变基的策略 [18]。在分析从 Git 挖掘的数据时，应该了解这些策略。图 5(b) 展示了

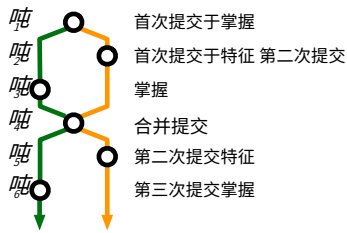


图 6. 分支后的 git DAG 掌握和特征 彼此融合。

如果 5 (a) 中的提交被展平（删除分支和合并提交）并重新排序，Alice 的 git 存储库将会是什么样子。

变基通常用于在将分支拉取到另一个仓库之前“清理”它。因此，git 仓库（尤其是稳定的官方项目仓库）中的历史记录可能无法反映到达该特定状态的实际情况。使用变基是挖掘卫星仓库（开发人员用来编写新功能的仓库）的原因之一，因为这些仓库可能包含完整的、未经修改的开发历史记录。

危险5： 您无法始终确定提交是在哪个分支上进行的。

虽然提交是在特定 Git 仓库的特定分支上明确创建的，但提交本身并不记录它创建于哪个分支。恢复这些信息可能很困难，甚至不可能。图 6 展示了交叉合并后的 Git DAG 结构。Alice 合并了一个名为特征到掌握，然后掌握

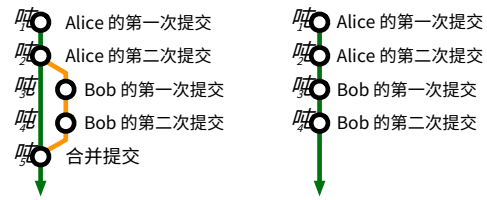
进入特征。这些合并共享一个合并提交，如图所示。

交叉合并之前的提交 $\mathcal{M}_4$ 都是两个后续分支的祖先。提交很少会标注其所属分支的名称和默认提交信息

合并提交是将分支“master”合并到特征，这并不能有效地区分它的父级。因此，如果存储库在某个时间被挖掘 $\mathcal{M}_6$ ，通常不可能知道合并之前哪些提交是在掌握（绿线前 $\mathcal{M}_4$ ）并且是在特征（橙线之前 $\mathcal{M}_4$ ）

危险6： 并不总是能够追踪合并的源，甚至不能确定是否发生了合并。

通常，当开发人员从远程 git 仓库的某个分支拉取提交到本地仓库时，该分支必须合并到当前的本地工作分支中。我们会喜欢了解合并的来源，无论是在



(a) 避免

(b) 执行

图 7. 如果 git 被明确告知避免快速合并 (a)，则 Alice 的存储库，而在拉取 Bob 的更改后执行快速合并 (b) 时则正常情况。

git 仓库及其分支的术语。在大多数情况下（但并非所有情况）这都是可行的。默认情况下，git 会为合并提交创建以下格式之一的日志消息。括号中的文本可能并非总是显示。

合并分支'分支名称' [进入分支名称] 合并 [分支'分支名称' 的] remote_repo_url

利用这些信息和 DAG 上提交之间的关系，可以看到哪些分支合并在一起以及在哪里合并。

无法检测合并的一种情况是快速向前合并如图 7 所示。Alice 向她的掌握，Bob 将其拉取到他的仓库，然后进行了两次提交。之后 $\mathcal{M}_4$ 如果 Alice 拉取 Bob 的更改，她的历史记录应该如图 7(a) 所示。然而，自从 Bob 拉取以来，她的仓库中没有任何变化，因此实际上不需要进行合并。除非明确告知 git 不要这样做，否则它会按顺序添加 Bob 的提交，并将 Alice 的 HEAD 版本“快进”到 Bob 拉取的最后一个提交。在这种情况下，不会创建任何合并提交。

提交消息中没有合并源的情况如下：

- 1) 如果两个分支的合并导致快速向前合并，则不会创建合并提交，因此日志中不会存在包含该字符串的日志消息。
- 2) 如果合并期间发生冲突，开发人员将解决冲突并使用可能不包含默认文本的日志消息提交其解决方案。
- 3) 如果开发人员重新设置一系列包含分支和合并的提交，结果可能会“扁平化”并且不包含合并提交。
- 4) 如果开发人员将合并提交的提交消息修改为默认合并消息以外的其他内容（这种情况不太可能，但有可能）。

图 8 说明了这种危险。 $\mathcal{M}_3$ Bob 从 Carol 拉取数据时，会创建一个合并提交；其提交消息指定了 Carol 仓库的 URL。假设 Alice 覆盖了默认的合并文本 $\mathcal{M}_5$ 当她从 Bob 拉取数据时。在 Alice 的 DAG 中，要知道哪个分支是 Bob 的，哪个分支是 Carol 的，并非易事。挖掘 Alice 的仓库，错过事实上，Alice 从 Bob 那里得到了



图 8. Alice 从 Bob 的仓库拉取数据后，Alice 的仓库。日志中没有明确记录 Alice 从 Bob 拉取的数据，因此可能会错误地推断 Alice 直接从 Carol 拉取了数据。

可能 **错误地得出结论** Alice 从 Carol 那里拉取了数据。任何基于代码库间拉取信息的分析都应该对这些问题敏感。

在上述前两种情况下，没有证据表明发生了合并。由于这些限制，无法确定有多少合并被遗漏，因此矿工在执行任何依赖于分支和合并的分析时必须谨慎。在后两种情况下，由于存在具有两个父提交的提交，因此仍然有证据，但是源信息可能会丢失。由于在这些情况下合并是已知的，因此可以衡量信息丢失。我们发现，对于 30 个 git 项目样本，启发式方法在 98% 的情况下有效。分析和结果的详细信息将在第 5 节中介绍。

承诺3：Git 在私人日志中记录纠正风险 3-6 所需的信息。

当 Bob 克隆 Alice 和 Ram 的仓库时，他对这些仓库历史记录的了解就会被遮挡。他无法确定 Alice 和 Ram 何时、从谁那里拉取了代码，何时以及如何进行 rebase 等等。然而，这些信息并不一定会丢失，即使它没有被 **琐碎地** 易于访问。Git 将有关快速合并、变基和拉取的信息存储在日志目录中。事实上，甚至可以在那里找到更多关于开发人员工作流程的信息

— 每次签出都会被记录下来，因此研究人员可以观察开发人员何时在分支之间切换。

这有望挖掘包含大量开发人员工作流程和项目历史信息的数据，但仅限于特定条件。矿工必须能够访问 **私人** 的她希望挖掘每个开发人员日志的存储库。因此，完全有可能编写利用这些信息的工具，供组织用于其内部项目。

在某些情况下，研究人员可能足够幸运，能够访问或复制一些他们希望挖掘的项目的私有开发者存储库。当这种机会出现时，研究人员必须意识到，公开的信息比来自

常规存储库克隆。

需要注意的是，Git 有一个清理命令，气相色谱。正常执行气相色谱不会干扰日志文件，但它有一个激进模式，会干扰日志文件。如果开发人员一直在运行 GC-- 积极一些日志条目可能已被删除。

危险7：可访问的数据可能仅包含成功选择的提交。

历史记录可能会因其他原因丢失或修改。在一种工作流程中，开发人员创建一个分支，其中包含需要其他开发人员拉取和审核的更改。审核完成后，如果更改未被接受，该分支可能会被销毁。或者，开发人员可以通过变基或在包含之前添加提交来修改分支。此过程类似于在 SVN 项目中提交补丁以供审核：该补丁可能会被拒绝，也可能最终提交（在可能的修改之后）。正如在基于 SVN 的项目中恢复补丁和审核过程很困难 [19], [20] 一样，提交的审核历史记录不会直接反映在 git 中。最初，我们天真地以为可以在 git 历史记录中看到更多审核过程。我们发现，结合使用邮件列表以及

签署人、审核人、和经 **fi** 测试领域
当它们存在时。

承诺4：签署和其他属性会形成“纸质记录”。

为了回应 SCO 提出的知识产权侵权指控，Linus Torvalds 增加了一项功能，允许人们在提交时“签名”，只需添加一行，例如由 John Doe <jdoe@foobar.com> 签名添加到提交消息的末尾。还有其他属性：ackedby、Cc、reportedby、reviewedby、和已测试。这些属性使用 - 明确添加到提交中佛罗里达州并使用易于解析的标准格式在提交日志消息中附加一行。提交在各个存储库之间移动、审核和测试时，可能会附加多个字段。

这些信息可以用来（例如）确定社区中的某些角色，例如审阅者或测试者。它还可以用来评估项目内的专业知识或重建社区的组织结构。我们将在第 5 节中展示这些数据的一种用途，即确定专业知识并可视化 Linux 社会组织层级。

到目前为止，我们观察到 Linux 内核之外的少数项目使用这些功能，但我们预计，随着采用更严格执行政策的项目采用 git，他们将利用这一附加功能来记录有关提交历史记录的信息。

承诺5: Git 明确记录了不属于核心开发人员的贡献者的作者信息。

在使用集中式 SCM 的传统 OSS 项目中，有一组明确的开发人员拥有源代码存储库的写权限。存储库日志会记录哪些开发人员进行了哪些更改。然而，OSS 项目严重依赖于志愿服务 [12]，许多明确的开发人员群体之外的社区成员也会以补丁的形式做出贡献，这些补丁通常会提交到开发邮件列表中。

如果补丁被接受，核心开发人员会将其提交到代码库，日志会显示该更改归因于该开发人员。理想情况下，我们希望能够将源代码更改归因于贡献更改的人员。遗憾的是，检测已提交补丁的应用情况很困难 [19]。在某些项目（例如 Apache）中，应用已提交补丁的开发人员通常会在提交消息中包含归属信息。然而，收集到的归属数据的准确性取决于项目本身、开发人员是否遵循惯例以及日志消息格式的标准化。Git 代码库拥有更准确的作者信息，因为它能够明确且自动地包含有关更改作者的信息。

基于 Git 的项目贡献者可以通过两种方式提交更改。任何拥有读取权限的用户都可以克隆公共 Git 仓库并提交到自己的副本。任何贡献者都可以请求项目开发者将更改从其自己的仓库拉取到主项目仓库。此外，Git 还提供了一种功能，可以将一次提交或一系列提交转换为特殊格式的补丁，该补丁可以通过电子邮件发送给项目开发者并应用于主项目仓库。即使是补丁，在提交到“官方”仓库时，作者信息也会被自动提取并保存。这种记录保存机制是 Git 提交具有以下特点的原因之一：*提交者*字段用于标识谁向存储库提交了更改，以及*作者*字段，用于标识更改的执行者。作者信息在提交从一个存储库传输到另一个存储库时与提交绑定。我们将在第5节中实证检验归因问题。

4. 开采吉特金矿脉

承诺6: 在 git 中全部元数据，尤其是历史记录，是本地的。

当开发人员基于现有存储库创建本地存储库时，远程存储库中的所有信息都会传输到本地存储库。过去

<pre># 包括 <math.h> int 添加 (int a) { ... }</pre>	<pre># 包括 <math.h> int sub (int a) { ... }</pre>
<pre>int sub (int a) { ... }</pre>	<pre>int 添加 (int a) { ... }</pre>
无效主 () {...} (a) Alice 的原作	无效主 () {...} (b) 由Bob修改

图 9. 文件的两个版本

在分析 OSS 项目时，我们需要分析存储库中每个文件的每个版本，我们依靠第三方工具（如 CVSSuck [21] 或 svn::mirror [22]）来创建本地镜像，以便我们可以快速签出文件，而不会增加官方公共存储库的负担。不幸的是，我们发现这些工具容易出错，有时还会造成损失。在某些情况下，我们需要联系项目维护者来请求他们的源代码存储库的副本。由于有关 git 存储库的所有信息都存储在本地，因此无需发出请求或使用第三方工具。所有分布式 SCM 都有此好处，而不仅仅是 git。当然，随着同一项目中存储库的分歧，元数据也会有所不同。

承诺7: Git 跟踪内容，因此它可以跟踪行在移动或复制时的历史记录。

Git 跟踪内容文件的重命名。这意味着它能够自动判断文件是否被重命名，因为内容保持不变，并且历史记录会跟随内容。在 SVN 中，开发人员必须显式重命名文件才能维护文件的历史记录。此外，Git 能够跟踪文件内部和文件之间的文本块。考虑图 9 中文件的两个版本。Alice 编写第一个版本。Bob 在移动函数时创建了第二个版本，而不会改变其内容。

如果svn 负责运行以显示谁引入了每一行，Bob 将被指示为函数中每一行的作者。然而，git blame -C -M，其中 -C 查找代码副本，-M 查找代码移动，这表明文件的每一行都是 Alice 编写的。Git 还会跟踪在文件之间移动的文本以及多次复制的文本，只要移动的行数超过某个阈值（我们观察到该阈值大约为 4 或 5）。当然，如果 Bob 在实现中修改了一行，即使是轻微的修改，git 也会将该行的作者身份分配给 Bob。Git 的来源分析不如最近引入的研究技术 [23] 那么准确，但与 SVN 相比，这是一个显著的改进。

承诺8: Git 速度更快, 而且通常比集中式存储库占用更少的空间。

Git 的设计目标是处理 Linux 规模的代码库, 并管理大量的贡献。由于元数据位于本地机器上, 因此访问日志或差异在本地执行, 没有网络延迟。此外, git 存储的是文件的完整(压缩)版本, 而不是一系列差异。这意味着检出文件是一个恒定时间操作, 与历史记录无关。

我们将整个 Eclipse CVS 仓库转换为 Git 格式, 发现日志访问和检出速度明显加快。使用 Git 检出任意(非连续)顺序的提交, 每次提交只需 1-2 分钟, 而使用 CVS 则需要 7-25 分钟。使用 Git 检出连续提交, 每次提交只需 0.5-0.8 秒, 而使用 CVS 检出我们服务器上的仓库和工作副本则需要 1-3 分钟。这使得一些之前非常繁琐的表单分析变得可行。

尽管 Git 的设计初衷是存储所有修订版本, 但其整体压缩策略占用的空间比 CVS 或 SVN 少得多。整个 Mozilla 仓库在 SVN 中存储时占用 12 GB。然而, 将仓库导入 Git 后, 占用空间缩减至 420 MB [24]。我们发现 Eclipse CVS 仓库占用 8.6 GB, 而 Git 仅占用 3.3 GB。Python 社区发现, 从 SVN 转换为 Git 后, 磁盘占用量从 1.3 GB 减少到 150 MB [25]。

承诺9: 大多数 SCM, 例如 CVS、SVN、Perforce 和 Mercurial (Hg), 都可以转换为 git, 并且分支、合并和标签的历史记录保持不变。

由于 git 在过去几年中的流行, 围绕它的开发社区已经开发了许多工具, 用于从最流行的 SCM 迁移到 git。

其中许多工具仍在积极开发中,

是官方 Git 发行版的一部分。例如, 有许多 git svn 充当 SVN 存储库和本地 git 存储库之间的双向通道的命令。克西我们已成功将整个 Eclipse CVS 存储库转换为 Git, 并包含标签和分支。我们还导入了 NetBeans 和 Mozilla-Central 的 Mercurial。(Hg) 将仓库转换为 Git 以及许多其他 SVN 仓库。通过能够将几乎所有仓库转换为 Git 格式, 我们能够获得 Git 的一些优势, 例如更好的来源检测、性能以及所有信息的本地副本。此外, 我们只需要编写 C 米只针对一种格式而不是多种格式的设计和分析工具。

3. 请参阅 <http://git.or.cz/gitwiki/InterfacesFrontendsAndTools> 了解完整信息。
李

Ruby on Rails 中按月份区分作者



图 10. 代码库日志报告的 Ruby on Rails 每月作者数量。蓝点表示 Rails 迁移到 git 的日期。

5. 分析

承诺 5 声称, Git 记录的信息和 workflows 能够更准确地分析作者。为了验证这一说法, 我们从使用 SVN 和 SVN 的项目中提取了数据。和 git 的使用情况, 并比较了作者信息。图 10 显示了按月对 Ruby on Rails 项目代码库做出更改的不同作者数量。图表上的点表示项目迁移到 git 的时间。虽然可能的转向 git 导致贡献者数量急剧增加, 但更有可能的是, 这种增加是由于 git 具有更优越的作者追踪功能。

根据风险 6, Git 中的日志消息记录了合并的来源。虽然可以启发式地从日志中恢复合并信息, 但这些信息可能不完整。我们通过挖掘 30 个使用 Git 的开源软件项目的数据, 实证检验了这些启发式方法的召回率, 这些项目列于 <http://git.or.cz/gitwiki/GitProjects> 尺寸从 9

作者 (disco) 已从另一个一千 (酒) 和包含迁移到超过 139,187 个 (Samba)。在项目 SCM 和 git 提交导入历史记录的情况下, 在过渡后我们只检查了 git。总共创建的合并 e 2,971 个合并 (IE 我有多个父级)。合并, 们忽略了无法检测的快速或者合并 flatte 能够检测到这些情况, 并产生了由于变基而导致。我们的启发式方一个可接受的范围, 以法在 2,909 个 f 中发现了合并, 占便根据来源执行类似的 97.9%。这似乎完全符合 st 用途。分析结果。然而, 研究人员在解释和报告合并时应该注意这一点。

我们假设使用 git 可能会影响 workflows, 从模式, 并在项目结束后集中式切换到 git 的切换更小、更开发提交频繁

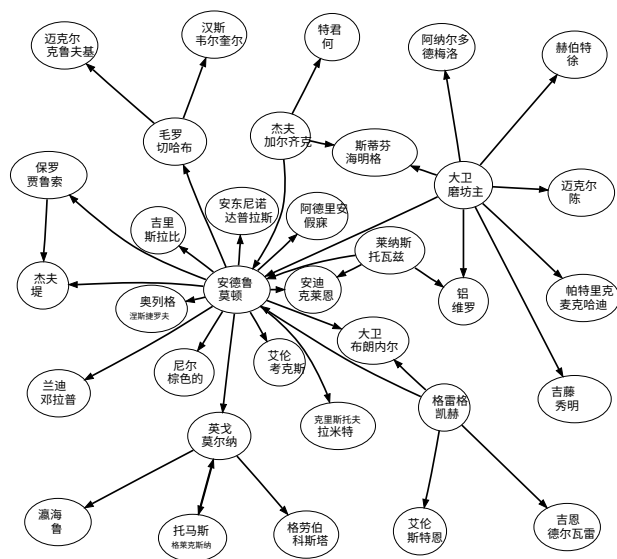


图 11. 签署Linux 内核中的网络。
来自一个到**b**表示一个立即签署提交**b**。

SCM。因此，我们对多个项目迁移到 Git 前后的提交大小（以新增和删除的行数衡量）进行了分析。虽然统计上显著，但差异幅度并不显著（每次提交仅少 2 行）。

为了评估承诺 4，我们研究了以下归因数据：签署包含在日志消息中。作为对 Linux 内核开发流程和工作流程的更广泛研究的一部分，我们发现（例如）大多数与非驱动程序相关的网络和 SPARC 架构相关的提交都由 David Miller 签署（分别为 69% 和 72%，这两个领域占其总签署量的 92%）。在 Miller 修改的所有文件中，87% 与网络相关，10% 与 SPARC 相关。Git 数据分析的这一结果证实了人们的传言：他既是这些领域的守门人，也是专家。

签名网络也使我们能够调查签名与贡献创作之间的关系。Torvalds 曾表示，他目前在 Linux 中的角色是集成者而非开发者。从量化角度来看，我们发现这一说法是正确的：他撰写了 632 次提交，执行了 4,317 次合并，并签署了 25,456 次提交。整个社区的签名数量与贡献创作数量之间存在中等非参数相关性 ($r=$.65, 页

001)。这表明，积极参与签署的开发人员（即层次结构中较高级别的开发人员）通常自己进行较少的开发，并且社区内存在差异化的角色。

全面研究“拉取网络”中开发人员签发文件的位置以及其在“拉取网络”中的位置，有助于识别社区成员承担的负载、他们的专业领域、地位或信任级别以及角色（审阅者 vs. 提交者）。我们根据开发人员签发提交的顺序重建了签发网络。图 11 展示了该网络的一部分，其中包括层级结构中的关键成员以及他们之间权重最高的边。

用于执行这些分析的数据完全从公共 Git 仓库中提取，并使用我们编写的工具存储在 PostgreSQL 数据库中。这些工具可供其他研究人员免费使用，网址为：git://github.com/cabird/git_mining_tools。

6. 结论

DSCM 承诺提供全新且实用的数据，帮助我们更好地理解软件流程。这些数据包括准确的作者信息；识别角色（例如审阅者、提交者和作者）；同一项目中不同开发人员的存储库内容差异；以及合并跟踪。与所有数据一样，这些数据必须谨慎处理。我们概述了挖掘 DSCM 数据时可能遇到的一些主要陷阱。值得注意的是，SVN 和 DSCM 之间提交和分支的语义有所不同；某些元数据（例如 DAG）可能由开发人员修改，并且通常无法确定提交发生在哪个分支上（这种风险使我们耗费了数天时间）。

我们计划利用这些数据来解答以下问题：1) 从集中式 SCM 切换到 DSCM 时，项目中的开发流程或沟通网络是否会发生变化？2) 使用 DSCM 是否会导致开发更加专注，但项目意识却有所下降？3) 开发人员团队是否会协同工作，并在一段时间内与“官方”代码库分离？我们希望本文能够增强其他研究人员收集、分析和解读这些数据以解答研究问题的能力。

参考

- [1] T. Zimmermann 和 P. WeißGerber, “预处理 CVS 数据以进行细粒度分析”，在《国际软件存储库挖掘研讨会论文集》，2004 年。
- [2] M. Fischer、M. Pinzger 和 H. Gall, “从版本控制和错误跟踪系统填充发布历史数据库” ICSM '03: 国际软件维护会议论文集. 美国华盛顿特区: IEEE 计算机协会, 2003 年, 第 23 页。
- [3] D. Cubranic、G. Murphy、J. Singer 和 K. Booth, “Hipikat: 软件开发的项目记忆工具”，《软件工程, IEEE 学报》, 第 31 卷, 第 6 期, 第 446-465 页, 2005 年。

- [4] T. Zimmerman、A. Zeller、P. Weissgerber 和 S. Diehl, “挖掘版本历史以指导软件变更”, *软件工程, IEEE 学报*, 第 31 卷, 第 6 期, 第 429-445 页, 2005 年。
- [5] B. Dagenais 和 MP Robillard, “建议框架演进的适应性变革”, *ICSE*, 2008 年, 第 481-490 页。
- [6] B. Fluri、H. Gall 和 M. Pinzger, “变化耦合的细粒度分析”, *源代码分析与操作, 2005 年. 第五届 IEEE 国际研讨会*, 第 66-74 页, 2005 年。
- [7] H. Gall 和 M. Lanza, “软件演化: 分析与可视化”, *2006 年国际软件工程会议论文集*, 第 1055-1056 页, 2006 年。
- [8] M. Pinzger、H. Gall、M. Fischer 和 M. Lanza, “可视化多个进化指标”, *2005 年 ACM 软件可视化研讨会论文集*, 第 67-75 页, 2005 年。
- [9] S. Kim、T. Zimmermann 和 K. Pan, “自动识别引入 Bug 的变更”, *第 21 届 IEEE 国际自动化软件工程会议 (ASE'06) 论文集-第 00 卷*, 第 81-90 页, 2006 年。
- [10] S. Kim、T. Zimmermann、E. Whitehead Jr 和 A. Zeller, “根据缓存历史记录预测故障” *第 29 届国际软件工程会议论文集*, 第 489-498 页, 2007 年。
- [11] S. Kim 和 MD Ernst, “通过分析软件历史确定警告优先级”, *MSR 2007: 软件存储库挖掘国际研讨会*, 美国明尼苏达州明尼阿波利斯, 2007 年 5 月 19 日至 20 日。
- [12] C. Bird、A. Gourley、P. Devanbu、A. Swaminathan 和 G. Hsu, “开放边界? 开源项目中的移民”, *MSR '07: 第四届软件存储库挖掘国际研讨会论文集*. 美国华盛顿特区: IEEE 计算机协会, 2007 年, 第 6 页。
- [13] C. Bird、D. Pattison、R. D'Souza、V. Filkov 和 P. Devanbu, “开源项目中的潜在社会结构”, *SIGSOFT '08/FSE-16: 第 16 届 ACM SIGSOFT 软件工程基础国际研讨会论文集*. 美国纽约州纽约: ACM, 2008 年, 第 24-35 页。
- [14] A. Mockus、JD Herbsleb 和 RT Fielding, “开源软件开发的两个案例研究: Apache 和 Mozilla”, *ACM 软件工程与方法学汇刊*, 第 11 卷, 第 3 期, 第 309-346 页, 2002 年 7 月。
- [15] B. Massey 和 K. Packard, “Regurgitate: 使用 GIT 进行 F/LOSS 数据收集”, *第一届软件开发公共数据国际研讨会*, 2006 年。
- [16] G. KroahHartman, “Linux 内核开发”, *Linux 研讨会*, 2007 年, 第 239-244 页。
- [17] J. Corbet, “补丁如何进入主线”, *Linux 每周新闻*, 2009 年 2 月 10 日。[在线]。可访问网址: <http://lwn.net/Articles/318699/>
- [18] “Git 管理。” [在线]。获取地址: http://kerneltrap.org/Linux/Git_Management
- [19] C. Bird、A. Gourley 和 P. Devanbu, “检测 OSS 项目中的补丁提交和接受情况”, *第四届国际软件存储库挖掘研讨会论文集*, 2007 年。
- [20] P. Rigby、D. German 和 M.-A. Storey, “开源软件同行评审实践: 以 Apache 服务器为例” *国际软件工程会议论文集*, 2008 年。
- [21] “CVSsuck。” [在线]。可访问网址: <http://cvs.m17n.org/~akr/cvssuck/>
- [22] “SVN-Mirror”。[在线]。获取地址: <http://search.cpan.org/dist/SVN-Mirror/>
- [23] G. Canfora、L. Cerulo 和 MD Penta, “从版本存储库中识别已更改的源代码行”, *第四届软件存储库挖掘国际研讨会论文集*, 2007 年。
- [24] “Git SVN 比较”。[在线]。获取地址: <http://git.or.cz/gitwiki/GitSvnComparsion>
- [25] “DVCS 后续: 管理 Python 仓库。” [在线]。获取网址: <http://ldn.linuxfoundation.org/blog-entry/dvcs-follow-what-about-managing-python-repository>